ASKUALALINK

HR Recruitment and Attendance Management System

PRODUCT REQUIREMENTS & ARCHITECTURE DOCUMENT

Version 1.0 · 2026 · Yohannes Bayeh

RATIONALE

AskualaLink replaces fraud-prone, manual HR operations with a single platform: attendance is cryptographically verified with GPS + Wi-Fi BSSID matching, candidate CVs are parsed and ranked by AI, top applicants are notified automatically, and payroll is calculated directly from verified attendance — cutting manual work and error across recruitment, attendance, and payroll into one enterprise system.

Project Name	AskualaLink HR Recruitment and Attendance Management System
Type	Full-Stack Enterprise HR Platform (Modular Monolith) — Private Enterprise System
Developer	Yohannes Bayeh,Henok Ashenafi,Yared Azene
Release Year	2026
Architecture	Modular Monolith — Recruitment · Attendance · Payroll · AI · Notification · Security modules
Primary UI Color	Enterprise Blue — #2563EB
Stack	Next.js · Tailwind CSS · Node.js/Express · PostgreSQL · Redis + BullMQ · Flutter · Grok API
Access	Private enterprise system — no public repository or live demo links included

CONTENTS

Section 1 — Executive Summary, Problem Statement & 4 Core Pillars

Section 2 — High-Level Functional Subsystems & User Roles

Section 2.5 — Application Architecture Diagram & System Flow

Section 3 — Technology Stack & Component Table

Section 4 — "4+1" Architectural View Model

Section 5 — Database Schema Specification

Section 6 — Security Architecture

Section 7 — Non-Functional Requirements & KPI Benchmarks

Section 8 — Document Control

SECTION 1 — EXECUTIVE SUMMARY, PROBLEM STATEMENT & 4 CORE PILLARS

1.1 Executive Summary

AskualaLink is a full-stack enterprise HR platform built as a modular monolith, unifying recruitment, attendance tracking, and payroll into a single system of record. It is designed for organizations that need to eliminate fraudulent attendance practices, accelerate hiring decisions with AI-assisted screening, and remove manual error from wage calculation — without the operational overhead of a distributed microservices architecture.

The platform is organized around four core pillars: identity-verified attendance, AI-driven talent acquisition, automated candidate engagement, and precision payroll management. Each pillar is implemented as a bounded module within the Node.js/Express monolith, sharing a single PostgreSQL data layer while remaining logically decoupled for maintainability.

1.2 Problem Statement

Problem	Impact
GPS spoofing and proxy (buddy) attendance clock-ins	Inaccurate presence records, payroll paid against fraudulent attendance
Manual resume screening for high-volume job postings	Slow time-to-interview, inconsistent evaluation criteria across recruiters
Communication lag with top-ranked candidates	Strong candidates go cold or accept competing offers before outreach occurs
Manual payroll entry from attendance records	Calculation errors, delayed pay slip generation, no audit trail on adjustments
No unified system linking attendance to pay	HR and payroll teams reconcile data manually across disconnected tools

1.3 The Four Core Pillars

Pillar	Capability
1. Identity-Verified Attendance	Anti-fraud clock-in combining Wi-Fi BSSID matching and GPS geofence validation to prevent spoofing and proxy attendance
2. AI-Driven Talent Acquisition	Grok AI parses uploaded CVs (PDF/DOCX) and scores candidates 0–100 against custom recruiter-defined prompts
3. Automated Candidate Engagement	HR selects a Top-N shortlist; NodeMailer dispatches automated HTML interview invitations to selected applicants
4. Precision Payroll Management	Verified attendance flags (Green/Red) drive automated presence-to-wage calculation and digital pay slip generation

SECTION 2 — HIGH-LEVEL FUNCTIONAL SUBSYSTEMS & USER ROLES

2.1 Functional Subsystems

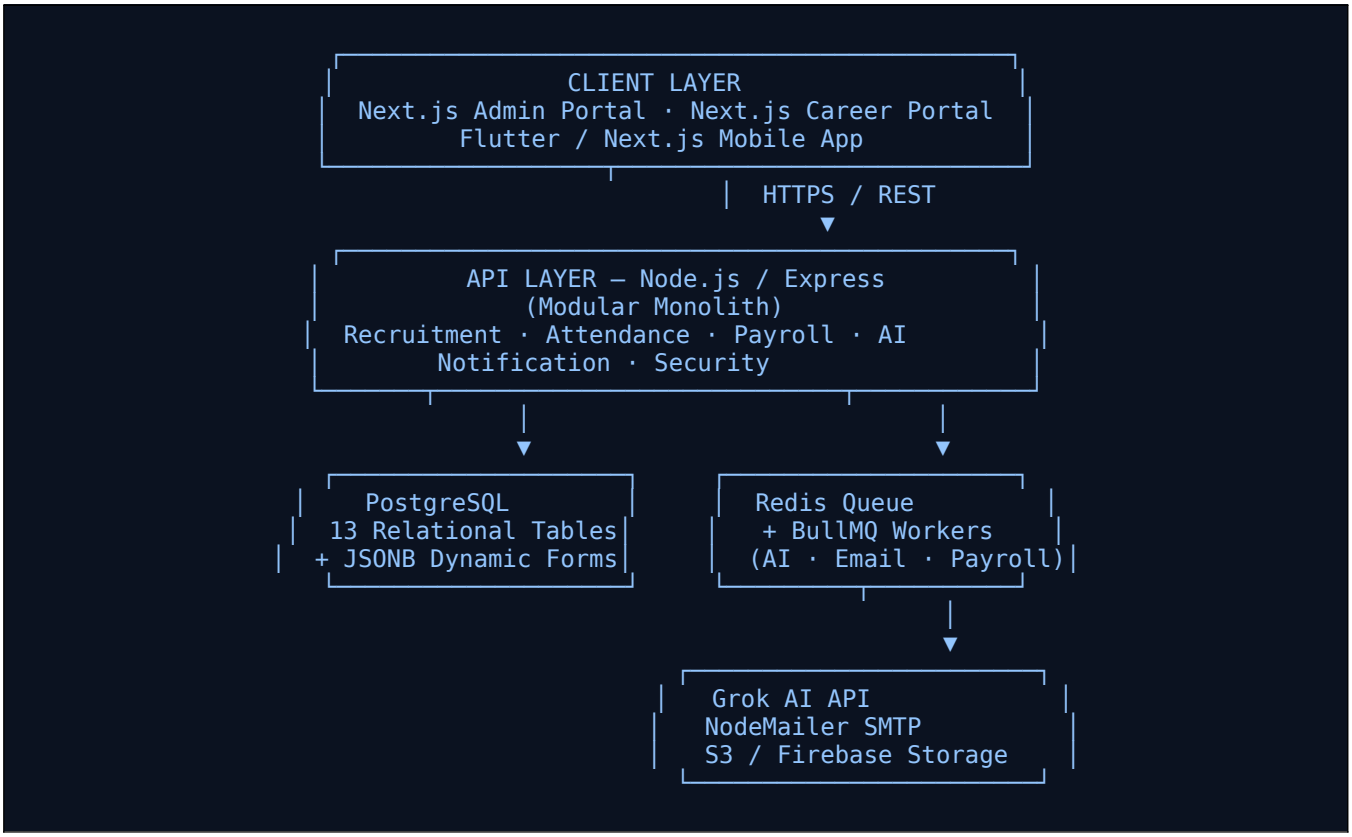
Subsystem	Responsibility
Attendance Engine	3-way validation of clock-in/out events; session management; anti-cheat enforcement
Recruitment Engine	Job posting management, applicant intake, CV storage, AI-driven scoring
Notification Engine	Top-N shortlist selection and automated HTML interview invite dispatch via NodeMailer
Payroll Engine	Presence-to-wage computation, deduction handling, digital pay slip generation
Security Module	Authentication, RBAC enforcement, encryption of sensitive fields, audit logging
AI Module	Grok API integration for semantic CV parsing and weighted candidate ranking

2.2 User Roles

Role	Access Scope
HR Admin	Full access — job postings, AI prompt configuration, shortlist selection, payroll generation, audit log review
Employee	Self-service attendance clock-in/out, personal pay slip viewing, personal profile management
Intern	Restricted variant of the Employee role — attendance clock-in/out and personal pay slip viewing only

SECTION 2.5 — APPLICATION ARCHITECTURE DIAGRAM & SYSTEM FLOW

2.5.1 System Application Architecture Diagram



2.5.2 System Flow Narrative

- Client requests originate from the Next.js Admin Portal, Next.js Career Portal, or the Flutter/Next.js mobile attendance app, and reach the API layer over HTTPS.
- The Node.js/Express modular monolith routes requests to the appropriate bounded module (Recruitment, Attendance, Payroll, AI, Notification, or Security) while sharing a single PostgreSQL data layer.
- Synchronous operations (e.g., attendance clock-in validation) are handled directly by the API layer for low-latency response.
- Asynchronous, resource-intensive operations — AI CV screening, bulk interview invite dispatch, and batch payroll generation — are offloaded to a Redis-backed BullMQ queue processed by background workers.
- Background workers call out to the Grok AI API for CV parsing/scoring, NodeMailer/SMTP for email dispatch, and S3/Firebase for CV and media storage, then persist results back to PostgreSQL.

> **RATIONALE — Why a Modular Monolith instead of Microservices?**
> *At the current scale (single enterprise deployment, 1,000+ employees), a modular monolith gives most of the maintainability benefit of clear module boundaries — Recruitment, Attendance, Payroll, AI, Notification, Security — without the operational cost of running, deploying, and monitoring a distributed system. Background-heavy work (AI scoring, bulk email, batch payroll) is isolated into a Redis/BullMQ queue rather than separate services, which keeps*

> *latency-sensitive paths like attendance clock-in fast while still allowing the queue-driven modules to scale independently by adding workers if load grows.*

SECTION 3 — TECHNOLOGY STACK & COMPONENT TABLE

Layer	Technology	Purpose
Admin & Career Portal	Next.js + Tailwind CSS	HR admin interface and public-facing career/job application portal
Mobile Attendance App	Flutter / Next.js	GPS & Wi-Fi based attendance clock-in/out on employee devices
API Layer	Node.js + Express	Modular monolith exposing REST endpoints across all functional subsystems
Primary Database	PostgreSQL	13 relational tables plus JSONB columns for dynamic form/prompt data
Async Processing	Redis + BullMQ	Background queue for AI screening, bulk email, and batch payroll generation
AI Engine	Grok API	Semantic CV parsing and weighted candidate scoring (0–100) against recruiter prompts
Email Delivery	NodeMailer	Automated HTML interview invitation dispatch to shortlisted applicants
File Storage	S3 / Firebase Storage	CV uploads and employee profile media
Deployment — Frontend	Vercel	Hosting for Next.js Admin Portal and Career Portal
Deployment — Backend	Render	Hosting for the Node.js/Express API and background workers

SECTION 4 — "4+1" ARCHITECTURAL VIEW MODEL

4.1 Logical View

The system is decomposed into six logical modules — Recruitment, Attendance, Payroll, AI, Notification, and Security — each owning a coherent slice of the PostgreSQL schema. Modules communicate through internal service calls within the monolith rather than network calls, preserving transactional consistency for operations that span modules (e.g., attendance status feeding directly into payroll calculation).

4.2 Process View

Two process types exist at runtime: request-handling processes within the Express API that serve synchronous, latency-sensitive operations (attendance clock-in, authentication, dashboard reads), and BullMQ worker processes that consume the Redis queue for asynchronous, longer-running work such as AI CV evaluation, bulk notification dispatch, and batch payroll runs. This separation ensures a slow AI scoring job cannot degrade attendance clock-in response times.

4.3 Physical View

The physical topology consists of a Next.js frontend deployment, a Node.js/Express backend deployment with attached BullMQ worker processes, a managed PostgreSQL instance, a managed Redis instance, and external managed services (Grok API, SMTP provider, S3/Firebase). Sensitive fields are encrypted at the database layer (AES-256) independent of the physical host.

4.4 Deployment View

Component	Platform	Notes
Admin & Career Portal (Next.js)	Vercel	Static/SSR frontend deployment
API + Background Workers (Node/Express, BullMQ)	Render	Backend API and queue worker processes
Database	Managed PostgreSQL	13 relational tables + JSONB dynamic form storage
Queue	Managed Redis	BullMQ job queue backing store

4.5 Scenario View (Representative Use Case)

Example scenario — "HR posts a role and hires within days": An HR Admin creates a job post and defines an AI scoring prompt (e.g., minimum CGPA, required certifications). Applicants submit CVs through the Career Portal; each CV is queued for Grok AI evaluation and scored 0–100 against the prompt. The HR Admin reviews the ranked list, selects a Top-N shortlist, and triggers automated HTML interview invitations via NodeMailer. Selected candidates receive interview details without manual email drafting, compressing the job-posting-to-interview timeline.

SECTION 5 — DATABASE SCHEMA SPECIFICATION

The system persists state across 13 core relational tables in PostgreSQL, supplemented by JSONB columns for dynamic form and prompt data. Primary and foreign key relationships are summarized below.

Table	Keys	Core Fields
users	id (PK)	email, password_hash, role, created_at, updated_at
employees	id (PK), user_id (FK → users.id)	full_name, national_id (AES-256 encrypted), department, position, hire_date, employment_status
office_wifi	id (PK)	office_location, bssid, ip_range, created_at
attendance	id (PK), employee_id (FK → employees.id), session_id (FK → attendance_session.id)	clock_in_time, clock_out_time, status (Green/Red), gps_lat, gps_lng, bssid_matched
attendance_session	id (PK), office_wifi_id (FK → office_wifi.id)	date, opened_at, closed_at
job_posts	id (PK), created_by (FK → users.id)	title, description, department, status, created_at
applicants	id (PK), job_post_id (FK → job_posts.id)	full_name, email, cv_file_url, applied_at
ai_prompts	id (PK), job_post_id (FK → job_posts.id), created_by (FK → users.id)	prompt_text, created_at
ai_evaluations	id (PK), applicant_id (FK → applicants.id), ai_prompt_id (FK → ai_prompts.id)	score (0–100), summary, evaluated_at
applicant_notifications	id (PK), applicant_id (FK → applicants.id)	notification_type, sent_at, status
payroll	id (PK), employee_id (FK → employees.id)	pay_period, gross_salary, deductions, net_salary, generated_at
payroll_view_status	id (PK), payroll_id (FK → payroll.id), viewed_by (FK → users.id)	viewed_at
audit_logs	id (PK), user_id (FK → users.id)	action, target_table, target_id, timestamp

SECTION 6 — SECURITY ARCHITECTURE

6.1 Identity & Access

Control	Implementation
Authentication	JWT token authentication with bcrypt password hashing
Authorization (RBAC)	Three roles — HR Admin, Employee, Intern — each scoped to defined endpoints and data views
Session Handling	Token-based sessions validated on every API request

6.2 Data Protection

Control	Implementation
Encryption at Rest	AES-256 encryption applied to sensitive HR fields — salary, national ID
Encryption in Transit	HTTPS enforced across all client-to-API and API-to-service communication
Network Geofencing	verifyWiFi.js middleware validates client IP and office router BSSID before accepting attendance events
Anti-Fraud Attendance Validation	3-way check — Employee SSO ID, GPS geofence, and Wi-Fi BSSID MAC address — with fake-GPS detection and hardware device binding
Audit Trail	Immutable audit_logs table records all administrative actions, payroll edits, and email dispatches

RATIONALE — Why GPS + Wi-Fi BSSID instead of a dedicated MDM solution?

A full mobile device management (MDM) rollout is heavier than a single-purpose attendance system needs — it requires enrolling every employee device and introduces additional IT overhead. Combining GPS geofencing with office router BSSID matching in verifyWiFi.js gives a two-factor location check (device location + network identity) that closes the two most common spoofing paths — fake-GPS apps and off-site proxy clock-ins — using only application-layer checks, with hardware device binding as a third layer against shared-credential abuse.

SECTION 7 — NON-FUNCTIONAL REQUIREMENTS & KPI BENCHMARKS

Category	Requirement / KPI
Recruitment Velocity	90% reduction in time from job posting to first interview invite
Attendance Performance	Clock-in response time under 500ms
Recruitment Performance	Applicant list load time under 2 seconds
Payroll Performance	Payroll batch generation completes in under 1 minute per batch
Scalability — Users	Supports 1,000+ employees on a single deployment
Scalability — Throughput	Handles 2,000 concurrent clock-ins per second

SECTION 8 — DOCUMENT CONTROL

Field	Detail
Document Title	AskualaLink HR Recruitment and Attendance Management System — Product Requirements & Architecture Document
Version	1.0
Date	July 27, 2026
Author	Yohannes Bayeh
Status	Final — Based on Verified Specifications
Distribution	Internal / Private Enterprise Reference